Distributed Multi-Model Analytics for E-commerce Data

AUCA Big Data Analytics — Final Project Technical Report

Student	Christian MANZI SHEMA · Student ID 101382
Course / brief	Big Data Analytics, AUCA — project brief dated 28 Oct 2026
Technologies	MongoDB 7.0 · HBase 2.1 (Thrift) · Apache Spark 4.0.5 (PySpark) · Python 3.13 · Docker
Date	9 November 2026 (submission due 11 Nov 2026, 11:59 am)

1. Introduction and objectives

This project builds a working analytics system in which MongoDB, HBase and Apache Spark each do the job they are best at, and then answers business questions that need all three together. It runs end-to-end on one Windows laptop: MongoDB and HBase in Docker, Spark in local mode on 8 cores. All numbers come from one reproducible run on 9 Nov 2026.

Part (weight)	What was built	Evidence
1. Data modeling (25%)	MongoDB document model: 5 collections, 9 indexes, 5 aggregation pipelines. HBase wide-column model: 2 tables, 7 query patterns	Notebooks 01–02; `hbase_shell_commands.txt`
2. Spark processing (30%)	Cleaning to partitioned Parquet; recommendation indicators; cohort analysis; 11 Spark SQL queries, 2 over live MongoDB + HBase data	Notebooks 03–04; `spark_sql_queries.sql`
3. Integration (25%)	Customer Lifetime Value and funnel conversion, each combining all three systems	Notebook 05
4. Visualization (20%)	8 static charts and auto-generated findings	Notebook 06; `outputs/figures`

2. Dataset and data-quality findings

The course generator was run at **medium** scale: 10% of the brief's session and transaction volume (401 MB of JSON). The full 2 million sessions need 15+ GB of RAM just to generate, because the script keeps every session in one Python list; the analysis code is identical at any scale. Only two changes were made to the generator: a `SCALE` setting, and writing each file in one call (much faster on Windows).

Entity	Records	Size	Notes
Users	10,000	1.8 MB	registered 2026-02-12 to 2026-08-11
Products / categories	5,000 / 25	1.6 MB	234 inactive, 163 out of stock; 3–5 subcategories per category
Transactions	50,000	21.9 MB	2026-08-11 to 2026-11-09 (91 days)
Sessions	200,000	376 MB	1,998,578 page views, in 2 files

Profiling the data **before** designing schemas found four facts that shaped every later decision:

1. **The `conversion_status` flag is unreliable:** 15,515 of 40,609 "converted" sessions (38%) have no transaction. Real purchases are always taken from transactions.
2. **Half of all orders have no session** (24,906 of 50,000), so the funnel can only use session-linked orders.
3. **Products have no `subcategory_id`** (the brief's sample shows one), so margin is estimated per category.
4. **Users have no age**, so segmentation uses purchase frequency, RFM, registration cohort and country instead.

No duplicate IDs, null keys or arithmetic errors were found: `subtotal − discount = total` holds for all 50,000 orders.

3. System architecture

MongoDB and HBase form the storage layer, each holding the data its model suits. Spark is the processing layer: it cleans the raw files, runs batch and SQL analytics, and joins live data from both stores for the integrated queries.

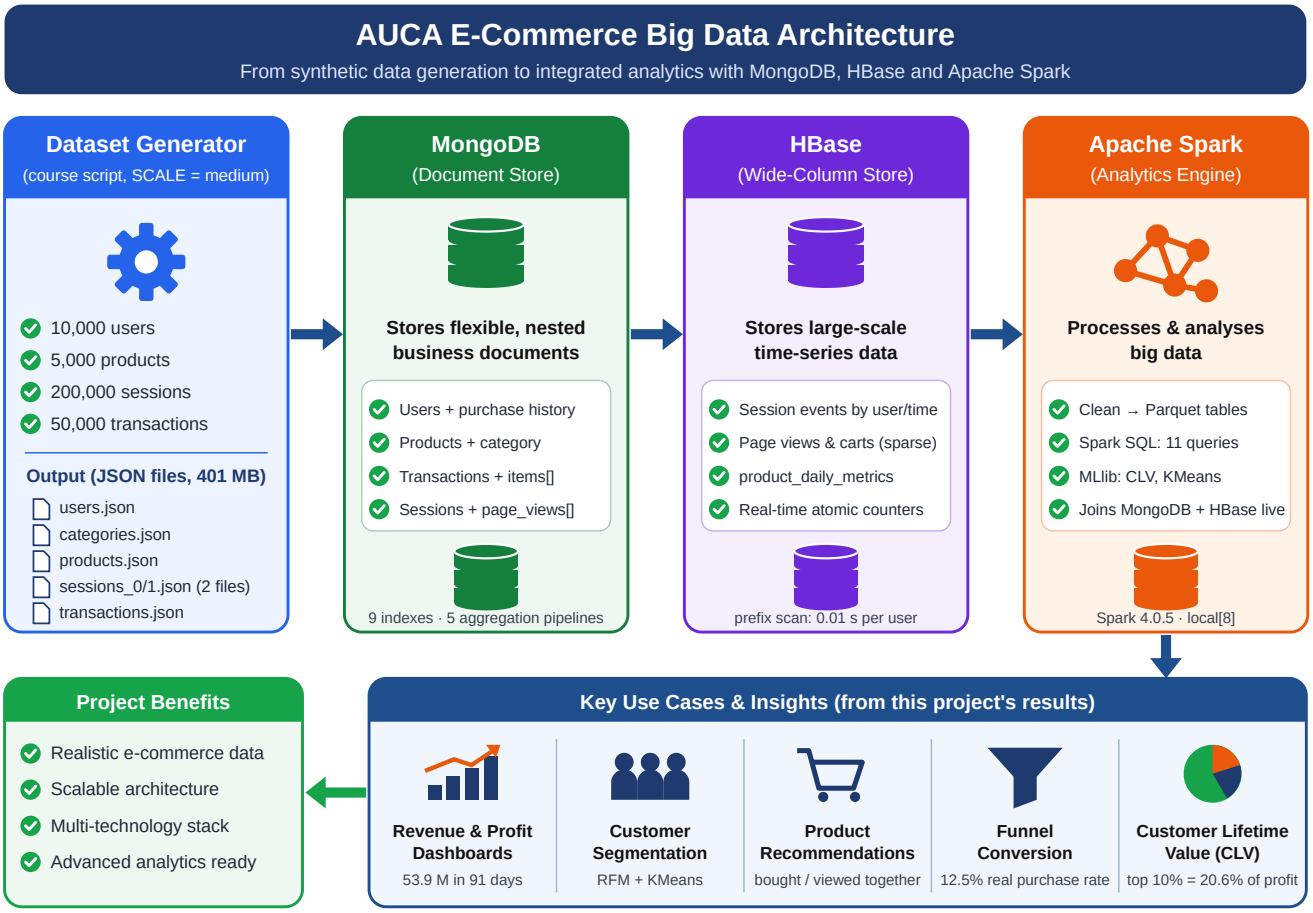


Figure A. System architecture (vector SVG; also delivered as `docs/architecture.svg`).

4. Data modeling decisions (Part 1)

Each store holds the data whose access pattern it serves best: MongoDB the rich, nested business entities queried in many different ways; HBase the high-volume, append-only session stream that is read by user and time.

Data	Store	Access pattern that decided it
Products + category; users + purchase summary	MongoDB	Read-heavy; filtered and segmented on many fields
Transactions + line items	MongoDB	Ad-hoc revenue analytics with `$unwind`, `$group`, `$lookup`, `$bucket`
Sessions (complete journeys)	MongoDB	Analyse a whole journey in one document (the brief's example query)
Session event stream	HBase `user_sessions`	"All sessions of user X, newest first" at any volume; sparse, variable columns
Product metrics per day	HBase `product_daily_metrics`	"Trend of product X over a date range"; atomic counters for streaming

Sessions are stored in both systems on purpose: MongoDB answers flexible journey questions; HBase serves the per-user lookup that must stay fast at billions of rows.

4.1 MongoDB document model

- **Embed what is read together and bounded:** a transaction's `items[]` and a session's `page_views[]`.
- **Denormalise small, stable attributes:** each product embeds `{category_id, name, avg_margin}` and each line item carries its `category_id`, so revenue by category needs no `$lookup`.
- **Replace dynamic keys with arrays:** `cart_contents: {"prod_00123": {…}}` cannot be indexed, so it becomes `cart_items: [{product_id, quantity, price}]`.
- **Store real dates** (`datetime`, not strings) so range queries and date operators use indexes.

- **Pre-compute summaries in the database:** two `$group` + `$merge` pipelines write `purchase_summary` and `preferences.top_categories` into each user, so segmentation reads 10,000 documents instead of joining 50,000 orders.

Indexes follow the query patterns (9 in total, e.g. `transactions(user_id, timestamp)`, multikey `items.product_id`). One customer's orders read **4 documents in 3 ms** with the index versus **50,000 documents in 27 ms** with a forced collection scan. Loading used batches of 5,000 with `ordered=False`; 200,000 sessions loaded in 24 s.

Aggregation pipeline (2 required, 5 built)	Brief category	Key technique	Result
Top-selling products	Popularity	`$unwind`, `$group`, `$lookup` after `$limit`	#1: 102 units, 46 buyers
Most-viewed products	Popularity	`$unwind viewed_products`	top: 193 sessions
Revenue and profit by category × month	Revenue	discount allocated pro rata; `$dateToString`	top category 2.60 M
Customers by purchase frequency	Segmentation	`$bucket` on `purchase_summary`	50% placed 4–6 orders
Daily revenue; conversion by device × referrer	Revenue / behaviour	conditional `$sum`	~20–21% in every group

4.2 HBase wide-column model

In HBase the row key is the schema. **`user_sessions`** uses `user_id # reverse_timestamp # session_id` with `reverse_timestamp = 9,999,999,999,999 − epoch_ms` (13 digits). User first keeps a user's sessions adjacent (one prefix scan); the reversed time sorts newest first, so "last 5 sessions" is a scan with `limit=5` and no sort; the session suffix keeps keys unique. Families: `meta` (duration, referrer, status), `device`, `geo`, `pv` (one column per page view: sparse, variable count) and `cart` (the product ID *is* the column name — natural in HBase, an anti-pattern in MongoDB). All families use `VERSIONS=1` and row Bloom filters; the shell script pre-splits the table into 5 regions. **`product_daily_metrics`** uses `product_id # yyyymmdd` with views, cart adds, purchases, units and revenue, kept current with atomic `counter_inc`.

Query (200,000 sessions loaded at ~1,540 rows/s)	Access path	Time
All sessions of one user / last 5 sessions	prefix scan (+ `limit`)	0.01 s
A user's sessions in a 14-day window; one product's 81-day trend	range scan on computed start/stop keys	0.02 s
Rebuild one full journey	point `get`	0.08 s
Key lookup vs full-table scan (100 users)	14.5 ms per user vs 4.84 s per scan	**~333× cheaper**

The trade-off is explicit: a new question such as "sessions by referrer" needs a full scan in HBase, so such questions go to Spark or to a second table keyed for them.

5. Spark processing pipelines (Part 2)

5.1 Batch pipeline

1. **NDJSON conversion** — the generator writes giant JSON arrays that Spark cannot split; one record per line lets Spark read in parallel.
2. **Explicit schemas** — no inference pass, and no silent type changes.
3. **Clean and validate** — drop null keys and duplicates, parse timestamps, standardise text, explicit defaults (`UNKNOWN` country, 0 stock); rule violations (`total = subtotal − discount`, `subtotal = Σ items`, duration > 0) are *flagged*, not deleted. A reconciled `is_purchased` flag from real transactions replaces `conversion_status`.
4. **Enrich** — price history, category margin (broadcast join), each line item's share of the order discount and estimated gross profit; funnel flags, cart value, hour and weekday per session.
5. **Flatten and write Parquet** — 1,998,578 page-view rows and 208,004 cart rows; sessions, page views and carts partitioned by `session_date`.

5.2 Product recommendation indicators

Support, confidence and lift are computed for every product pair with a basket self-join (`a.product_id < b.product_id`). "Bought X also bought Y" (50,000 orders) kept 351 pairs seen ≥ 2 times; "viewed together" (171,920 sessions) kept 2,130 pairs seen ≥ 3 times, turned into a top-5 recommendation table per product with a window function. A **null-model check** refilled the same baskets with random products: it produced 1,652 pairs with median lift 28.3, against 2,130 and 24.7 for the real data. The same order of magnitude means **the generator has no built-in product affinity** — the pipeline is correct, but there are no real relationships to find.

5.3 Cohort analysis

Users are grouped by registration month (Feb–Aug 2026) and their spending measured in three **equal 30-day periods**, so every cohort has the same exposure. About **80% of every cohort buys in every period**, spending 1,688–1,879 per user (period average 1,765 → 1,826). There is no cohort effect (Figure 6).

5.4 Spark SQL analytics

#	Question	SQL techniques	Key result
A1	Top 3 products per category	CTE + `RANK() OVER (PARTITION BY …)`	75 rows
A2	Daily revenue trend	`AVG … ROWS 6 PRECEDING`, `LAG(…,7)`	stable near 600 k/day
A3	RFM scoring	`NTILE(5)` × 3, `CASE`	2,165 Champions (avg 7,865); 2,391 At risk (3,083)
A4	Funnel by device	conditional aggregation	real 12.5% vs platform flag 20.3%
A5–A6	Category conversion; views vs sales	3 CTEs joined; `LEFT JOIN` + `COALESCE`	view-to-cart ~30%; r = 0.70
A7–A9	Weekday × hour; payment × status; countries	joins, `PIVOT`	apple_pay/crypto only "completed"
B1	Does engagement (HBase) predict spend (MongoDB)?	cross-system join + `NTILE(3)`	high vs low: 5,885 vs 4,933 (+19%)
B2	Loyal buyers who stopped visiting	cross-system join + `DATEDIFF`	re-engagement list

B1 and B2 run on data loaded live from MongoDB (PyMongo) and HBase (Thrift scan of the `meta` family only) by `src/federation.py`; the notebook prints `MongoDB (LIVE) + HBase (LIVE)`. If a store is down it falls back to Parquet and labels the result "SIMULATED". All 11 queries are in `outputs/spark_sql_queries.sql`.

Optimisations, measured: Parquet answered a sessions-per-device query in 1.06 s vs 3.84 s for NDJSON (3.6× faster); partitioning by date made a one-week query read only 7 folders (`PartitionFilters`); small dimensions are broadcast (`BroadcastHashJoin`, no shuffle of the large side); 16 shuffle partitions + adaptive execution instead of 200; reused DataFrames are cached.

6. Integrated analytics workflows (Part 3)

Two integrated queries were built (one required). Both read MongoDB and HBase **live** and use Spark for the joins and computation neither store can do alone.

6.1 Query A — Customer Lifetime Value

Business question: which customers will be worth the most over the next 12 months, and does browsing engagement add anything beyond past spend? **Workflow:** (1) MongoDB returns user profiles with the pre-computed `purchase_summary` (0.9 s); (2) HBase returns per-user sessions, duration and page views from a scan of the small `meta` family (9.6 s); (3) Spark joins both with line-item gross profit; (4) Spark scores 12-month CLV; (5) MLlib validates the signal on a time split; (6) MLlib KMeans segments customers.

$$\text{CLV}_{12m} = (\text{gross profit observed} \div \text{months observed}) \times \Sigma_{t=0..11}\, r^t / (1+d)^t, \quad r = 0.90,\ d = 1\% \rightarrow \text{multiplier } 6.88$$

- Projected 12-month profit is **30.9 M**; the top 10% of customers hold **20.6%** of it, the top 30% hold 49%.
- KMeans chose k = 2 (silhouette 0.456 vs 0.391 for k = 3): **Engaged big spenders are 41% of customers but 61% of projected profit** (6.9 orders, 8,041 spend on average).

- **Honest validation:** linear regression and a 100-tree random forest beat the run-rate baseline (RMSE 1,655 vs 2,054) but $R^2 \approx 0$, with flat feature importances. Engagement carries no extra signal about next month's spend in this synthetic data; on real data, this same test decides whether to trust an engagement-based CLV.

6.2 Query B — Funnel conversion

Business question: how many sessions view, add to cart, reach checkout and really buy; where is the biggest loss; and how much money is at stake? **Workflow:** journeys come from the session data (HBase `pv`/`cart` families; the all-user funnel is a full-scan aggregation, so it runs in Spark); purchases come from MongoDB `transactions.distinct("session_id")`. Spark's purchased-session set equals MongoDB's exactly (25,094 = 25,094). Each funnel stage requires the one before.

- **12.5%** of sessions end in an order; the biggest loss is **cart → checkout** (72,134 sessions, 44.6% step conversion).
- The platform flag reports 40,609 conversions against 25,094 real orders: it **over-reports by 62%**.
- **Money at stake:** 72.0 M in abandoned carts and **12.3 M in failed checkouts** (flagged converted, no order — likely stock-outs), against 20.0 M actually purchased.
- Device and referrer rates span only 12.1–12.8%; with ~40,000 sessions per group this is noise.

Performance: HBase reads only the light `meta` family and MongoDB serves a pre-aggregated summary, so the expensive joins happen once, in Spark; extracted DataFrames are cached because MLlib iterates over them.

7. Visualizations and business insights (Part 4)

Eight static charts (Matplotlib/Seaborn, 200 dpi) were produced; the six most informative are shown. Each insight sentence was generated from the result tables.

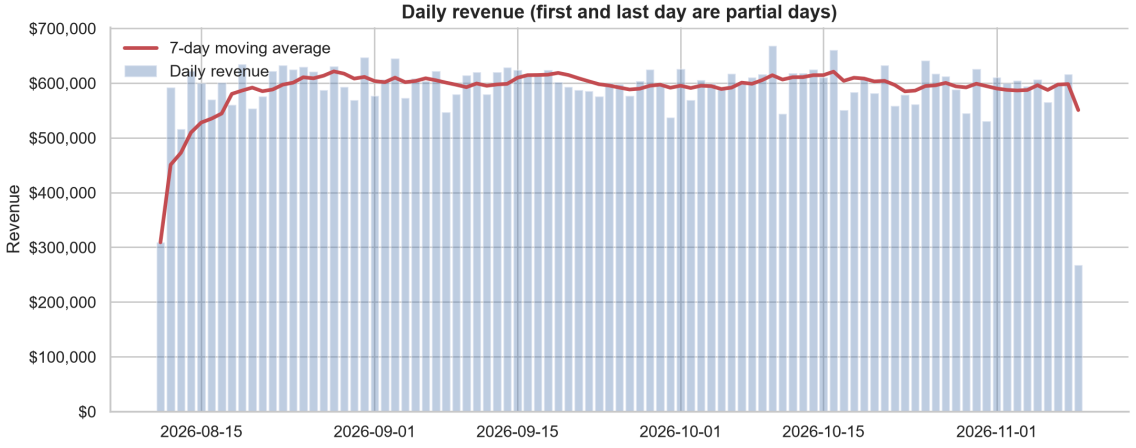


Figure 1. Daily revenue and 7-day moving average — 53.9 M over 91 days, steady at ~599 k/day (5% day-to-day variation; edge days are partial).

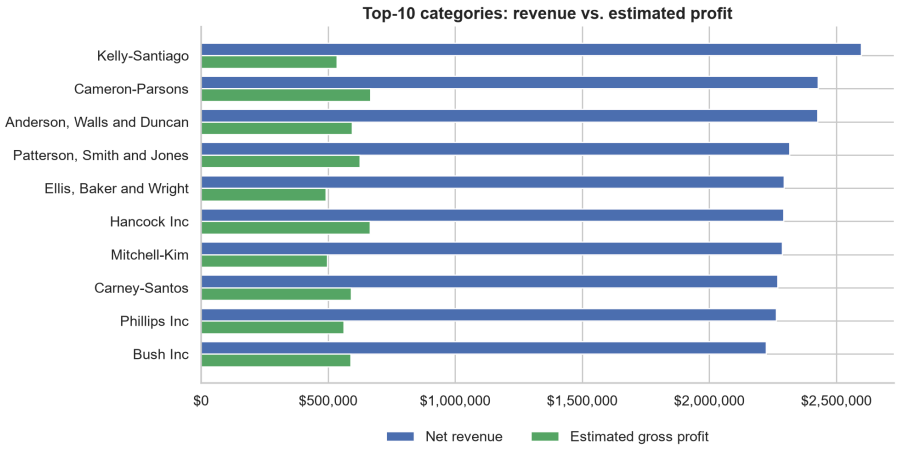


Figure 2. Top-10 categories: revenue vs estimated profit.

Category profit ≠ revenue. Kelly-Santiago leads revenue (2.60 M) but Rodriguez-Graham earns the most estimated profit (748 k); margins range from 18% to 36%. Promotions and shelf space should follow profit.

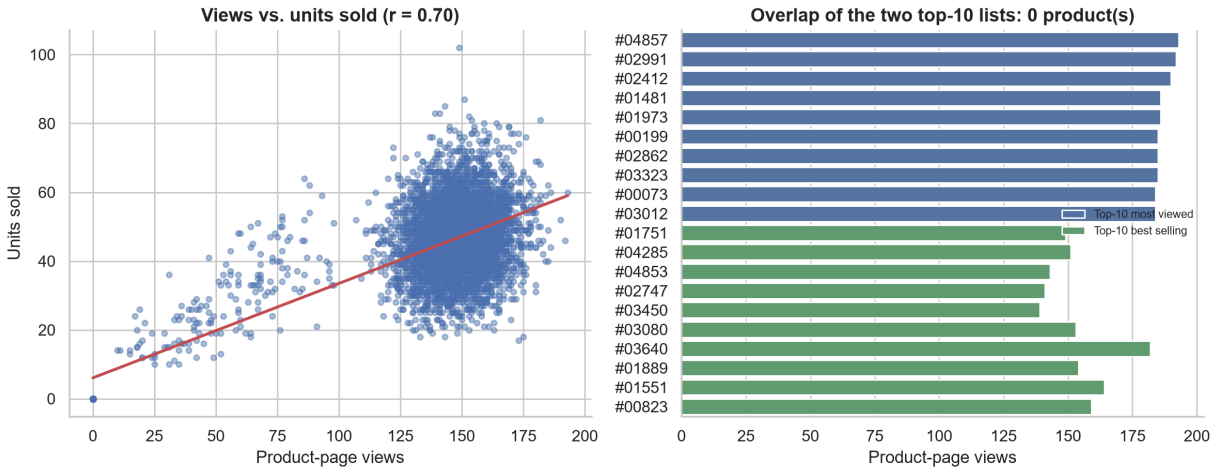


Figure 3. Views vs units sold (r = 0.70); none of the 10 most-viewed products is a top-10 seller.

Popular pages are not the best sellers. Products with many views but few sales are candidates for a price or product-page review.

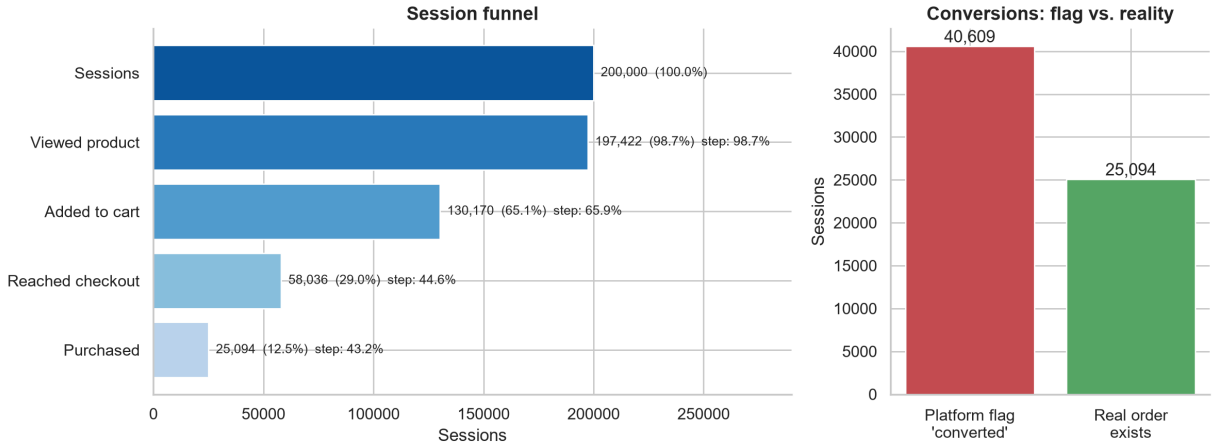


Figure 4. Strict session funnel and the platform's 'converted' flag vs real orders.

The funnel leaks most between cart and checkout, and the platform's flag over-states conversion by 62%. Reporting must use real orders; stock availability at checkout is the first fix.

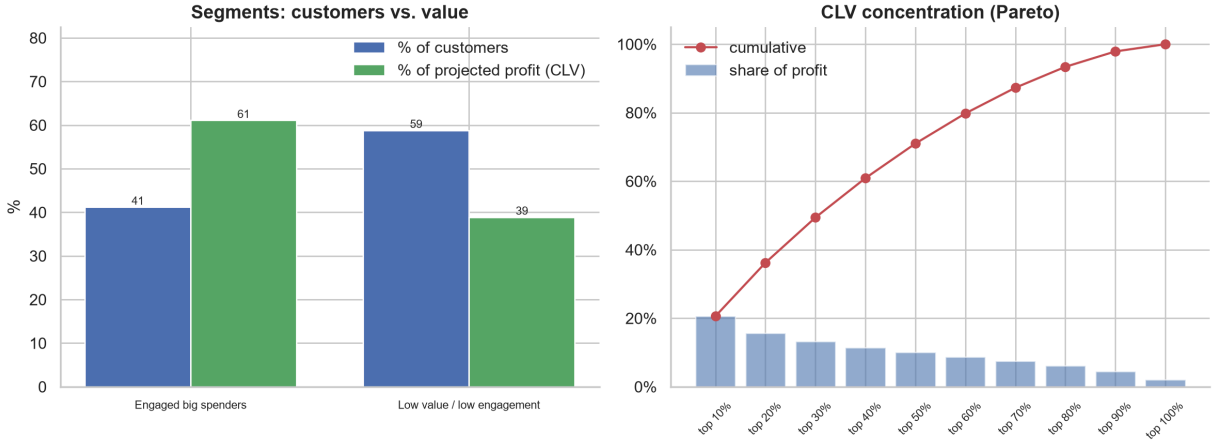


Figure 5. Customer segments vs share of projected profit, and CLV concentration by decile.

Value is concentrated but not extreme: 41% of customers hold 61% of projected profit, but the top 10% hold only 20.6% (not an 80/20 business), so broad retention still matters.

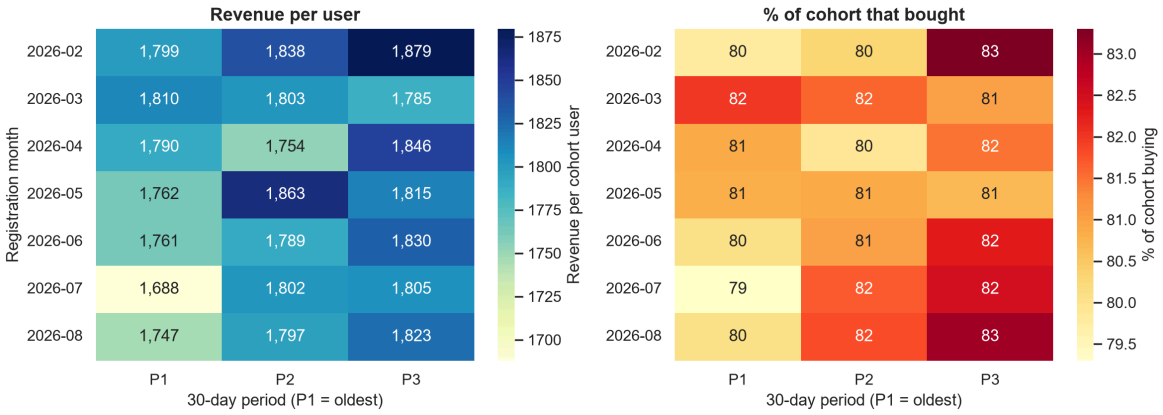


Figure 6. Cohort heat-maps: revenue per user and share of cohort buying, by 30-day period (note the narrow colour scales).

No cohort under-performs: 79–83% of each cohort buys in every period. The remaining two charts (purchase rate by channel; traffic by weekday × hour) show only noise-level differences — channel rates within 0.7 points and hourly variation of 4% vs 3% expected by chance — so **no channel or timing budget should move on this evidence**.

Recommendations

1. Measure conversion from real orders, never the platform flag (62% over-reporting).
2. Fix stock availability at checkout: 12.3 M of cart value was lost after customers had decided to buy.
3. Target retention at Engaged big spenders (61% of profit) and the 2,391 RFM "At risk" customers.
4. Review price and pages of high-view, low-sale products; manage categories on profit, not revenue.

8. Scalability, limitations and future work

Component	Scales by	Watch-out at 10× volume
MongoDB	Sharding (`transactions` on hashed `user_id`; `sessions` on `{user_id, start_time}`); per-shard aggregation	Cross-shard `$lookup` is costly — hence denormalised categories
HBase	Regions split by key range across region servers; per-user scans stay in milliseconds at any size	Sequential IDs concentrate writes: pre-split (done) or salt `hash(user_id) % N`
Spark	More executors; partitions follow the Parquet files; same code	Pair self-joins grow with basket size: cap or sample baskets
Ingestion	Parallel loaders; HBase bulk-load from HFiles; official Spark connectors	Generator must stream records (2 M in one list needs 15+ GB)

Streaming: the demonstrated HBase `counter_inc` on `product_daily_metrics` is the write path for a Spark Structured Streaming job reading page-view events from Kafka, so batch and stream share one table.

Limitations. (1) Synthetic, independently random data: the null-model affinity result, flat cohorts, $R^2 \approx 0$ and even channels are properties of the data, not method failures, and are not findings about real customers. (2) No `subcategory_id` or age. (3) 10% of the brief's volume, a configuration choice for an 8 GB laptop. (4) 91 days cannot show churn, so CLV rests on a stated 90% retention assumption; a sensitivity check at 80% and 95% is advised. (5) Single-node timings show relative, not cluster, cost. (6) Half the orders have no session.

Future work. Replace the Python bridges with the `mongo-spark` and `hbase-spark` connectors; add the Structured Streaming job; train collaborative filtering (Spark ALS) on richer data, keeping the null-model check; fit a churn model on a longer history.

9. Conclusion and technology selection

Analytical task	Technology	Reason
Revenue, popularity, frequency segments	MongoDB aggregation	Nested `items[]`, indexed `$unwind/$group/$bucket`, no ETL
A customer's latest sessions; a product's daily trend; live counters	HBase	Row-key design → prefix/range scans in 0.01–0.02 s; atomic increments
Cleaning 2 M page views, pair mining, cohorts, SQL reporting	Spark / Spark SQL	Full-data passes, self-joins, window functions, `PIVOT` in parallel
CLV and funnel across all stores	Spark + MongoDB + HBase	Only Spark can join both stores and run MLlib on the result

The three technologies proved complementary rather than interchangeable. The analysis also checked itself: it reconciled an unreliable conversion flag against real orders, tested product affinities against a random baseline, and validated the CLV signal on a time split. Where the synthetic data could not support a conclusion, the report says so.

Reproducibility. Repository: *GitHub link to be added*. Run `docker compose up -d`, install `requirements.txt` (Python 3.10–3.13, Java 17), then notebooks 00 → 06 in order (about 20 minutes in total). Supporting files: `hbase/hbase_shell_commands.txt`, `outputs/spark_sql_queries.sql`, `outputs/key_findings.md`, `outputs/tables/*.csv`, `docs/*.xlsx`.